ACADEMIC PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A LIGHTWEIGHT LINUX CONTAINER RUNTIME FROM SCRATCH IN C

Operating System Virtualization, Cgroups v2, Concurrency, and Memory Protection

Course: Operating Systems (CSE Core)

Implementation: C99 / POSIX System Calls & Linux Kernel 6.x

Team Members:

1. **Tharakesha M** (Reg No: **24BCE2538**)

2. **Mathu Metha S** (Reg No: **24BCE2805**)

Department: Department of Computer Science & Engineering

Academic Year: 2026

Project Codebase: `mini-container` (Zero External Runtime Dependencies)

1. ABSTRACT

Modern cloud computing and DevOps infrastructure depend fundamentally on container virtualization platforms such as Docker and Kubernetes. However, containers are often perceived as complex abstractions without an understanding of the underlying kernel architecture.

This project designs, implements, and evaluates `mini-container`, a standalone, lightweight container runtime written from scratch in pure **C99** with zero third-party libraries. By directly interfacing with Linux kernel subsystems—specifically **Linux Namespaces** (`CLONE_NEWPID`, `CLONE_NEWUTS`, `CLONE_NEWNS`, `CLONE_NEWIPC`), **Unified Cgroups v2** (`memory.max`, `cpu.max`, `cpu.weight`, `pids.max`), and **VFS filesystem sandboxing**—the runtime achieves complete hardware resource confinement and process tree isolation.

The project experimentally demonstrates all five Course Outcomes (CO: 1 to CO: 5) of the Operating Systems syllabus, including system call dispatch, Completely Fair Scheduling (CFS), POSIX semaphore synchronization, demand paging accounting (minor and major page faults), and kernel Out-Of-Memory (OOM) Killer execution.

2. LITERATURE SURVEY: CONTAINERS VS. VIRTUAL MACHINES

Dimension	Virtual Machines (VMs)	Operating System Containers (mini-container)
Virtualization Layer	Hardware Level (Hypervisor)	Kernel Level (Namespaces & Cgroups)
Guest OS Kernel	Requires separate guest OS kernel	Shares single host Linux kernel
Boot Latency	Tens of seconds to minutes	Sub-second (10 - 20 milliseconds)
Memory Footprint	Gigabytes (Guest OS memory overhead)	Megabytes (Confined purely to user application)
Performance	Incurs hardware translation penalties	Near-native bare-metal execution speed

3. COURSE OUTCOMES (CO) ALIGNMENT MATRIX

Course Outcome	Syllabus Topics	Implementation in mini-container
CO 1: OS Principles	System calls, Protection modes, PCB, Process creation	`clone()` syscall with isolated stack, `waitpid()`, PCB lifecycle
CO 2: CPU Scheduling	Preemptive quotas, CFS scheduling, Proportional share	Cgroups v2 `cpu.max` (quota/period) and `cpu.weight` fair share contest
CO 3: Concurrency & IPC	Synchronization primitives, Semaphores, IPC	POSIX `pipe()` barrier, Producer-Consumer with `sem_t` and shared memory
CO 4: Memory Management	Virtual memory ceilings, Page faults, Demand paging	Cgroups v2 `memory.max`, `pgfault` tracking, kernel OOM Killer `SIGKILL`
CO 5: Virtualization & Filesystems	Container virtualization, VFS, Private mounts	`CLONE_NEWNS` mount isolation, `chroot()`, private `/proc` mount

4. SYSTEM ARCHITECTURE & EXECUTION FLOW



5. EXPERIMENTAL RESULTS & EMPIRICAL VALIDATION

Experiment	Configuration	Observed Output	Syllabus Concept Verified
1. Process Isolation	`--hostname isolated-box ./tests/test_info`	Process runs as **PID 1**; host processes hidden	CO: 1 & CO: 5 (Namespaces)
2. Memory Limit & OOM	`--memory 30M ./tests/test_memory 60`	Allocated up to 30MB; terminated by **SIGKILL (signal 9)**; logged **7,705 Page Faults**	CO: 4 (Virtual Memory & Demand Paging)
3. CPU Quota Throttling	`--cpu 40 ./tests/test_cpu 3`	Compute loop clamped at **37.4%** CPU usage	CO: 2 (CFS Scheduling Quotas)
4. CPU Fair-Share Contest	Contest: Weight 800 vs Weight 200 on Core 0	CPU time: **2.78s vs 0.69s** (Exact **4.02 : 1** ratio)	CO: 2 (Proportional Share Scheduling)
5. Concurrency & IPC	`./tests/test_ipc_sync`	10 items produced/consumed cleanly via `sem_t`	CO: 3 (Semaphores & Shared Memory)
6. Fork Bomb Defense	`--pids 10 ./tests/test_forkbomb 50`	Fork #10 blocked with **EAGAIN (errno 11)**	CO: 1 & CO: 3 (Process Table Protection)

6. VIVA VOCE DEFENSE CHEAT SHEET (TOP FACULTY QUESTIONS)

> **Q1: Why did you use `clone()` instead of standard `fork()`?**
>
> *Answer:* `fork()` duplicates the process in the exact same namespaces. `clone()` accepts flags like `CLONE_NEWPID` and `CLONE_NEWUTS`, instructing the Linux kernel to instantiate the child directly inside brand-new, isolated namespaces.

> **Q2: Why did your memory test trigger `SIGKILL` instead of `malloc()` returning NULL?**
>
> *Answer:* Linux uses optimistic memory allocation (overcommit) and demand paging. `malloc()` initially reserves virtual address space without committing physical pages. When `memset()` touched the pages, memory demand crossed `memory.max`. With swap disabled, the kernel Out-Of-Memory (OOM) Killer stepped in and dispatched an uncatchable `SIGKILL`.

> **Q3: What is the purpose of the POSIX pipe between parent and child?**
>
> *Answer:* It acts as a barrier synchronization mechanism (CO: 3). Without it, a race condition occurs where the child might call `execvp()` before the parent finishes creating the cgroup and writing the child's PID into `cgroup.procs`.

> **Q4: How did your project prove CPU scheduling fairness?**
>
> *Answer:* We ran two containers simultaneously pinned to CPU Core 0 with CFS weights of 800 and 200. The high-weight container received 2.78 seconds of CPU time while the low-weight container received 0.69 seconds, achieving an exact 4:1 execution ratio matching theoretical scheduling weights.

> **Q5: What are Minor Page Faults and how did you verify them?**
>
> *Answer:* Minor page faults occur when virtual pages are mapped to physical RAM without requiring disk I/O. We parsed `/sys/fs/cgroup/minicontainer-[PID]/memory.stat` and recorded over 7,700 minor page faults during memory allocation, proving demand paging in the memory management unit.

7. CONCLUSION

The `mini-container` project demonstrates that robust operating system virtualization, process isolation, CPU scheduling, concurrency synchronization, and memory protection can be achieved purely using native Linux kernel subsystems in C with zero external runtime overhead. The project completely satisfies all Course Outcomes (CO: 1 to CO: 5) and offers a production-grade systems engineering demonstration for academic evaluation.